

Arquitectura de Agentes (Agentic Systems) — Análisis completo de la Sesión 8

Fuente base: *Agentic Systems — Arquitectura de un Agente* — Módulo 4 (Agentes Cognitivos), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora (Technology Architect on GenAI (*Generative Artificial Intelligence, IA generativa*); +15 años de experiencia; Credicorp, BCP, TCS).

Complementado con: investigación propia sobre la definición formal de agente racional (Russell & Norvig), el framework ReAct, los seis pasos de **CRISP-DM** (*Cross-Industry Standard Process for Data Mining, proceso estándar inter-industrial para minería de datos*) aplicados a agentes, una tabla comparativa ampliada de frameworks de orquestación (LangGraph, CrewAI, AutoGen, Semantic Kernel, Strands Agents, smolagents), y el contexto de mercado 2025-2026 (Gartner, McKinsey, MIT).

Propósito de este documento: esta es la sesión donde el arco de Módulo 3 (contratos → razonamiento → modularización con guardrails/HITL (*Human-In-The-Loop, humano-en-el-bucle*)) se convierte formalmente en **arquitectura de agente**: qué componentes tiene, cómo se combinan, y cómo se clasifica un sistema agéntico según quién controla el plan y la ejecución.

0. Dónde se ubica esta sesión — el salto de "pipeline" a "agente"

Módulo 3, Clase 7 (cierre):

"Un pipeline con contratos + gates + guardrails + HITL + observabilidad ES, literalmente, el esqueleto de un agente de producción.

El único salto que falta: dejar que el LLM dirija el orden."



Módulo 4, Sesión 8 (esta):

Ese salto se formaliza: se define QUÉ es un agente, QUÉ NO lo es, y con QUÉ COMPONENTES se construye uno (comunicación, contexto, entorno, autonomía, criticidad).

La sesión tiene dos bloques temáticos consecutivos, visibles en las dos diapositivas de "Agenda" del material (páginas 10 y 11/25):

Bloque	Contenido	Entregable de laboratorio
A — Fundamentos	Conversational Framework, definición de Agente, Arquitectura de Componentes, Dimensiones Funcionales	<i>Agent Profile Card</i> → <i>System Prompt</i>
B — Clasificación y orquestación	Clasificación de sistemas agénticos, orquestación determinista/no determinista, frameworks de investigación globales (Anthropic, OpenAI, Gartner, McKinsey), referencia de ciclo de vida (SDLC , <i>Software Development Life Cycle</i> , ciclo de vida de desarrollo de software) vía CRISP-DM	Borrador inicial en Draw.io + <i>mocking</i> de agentes (sin librerías de agentes, "a mano")

1. Contexto de apertura — el estado real de la industria en 2026

Antes de definir nada, el material presenta un panorama deliberadamente contradictorio, con dos lecturas simultáneas:

Señales de sobre-expectativa (hype):

- *"MIT report: 95% of generative AI pilots at companies are failing"* (Fortune, ago-2025) — la mayoría de los pilotos de IA generativa en empresas no llegan a producción o no generan valor medible.
- Un meme sobre "Apagón Global de IA" y viñetas sobre regulación — la conversación pública oscila entre pánico regulatorio y euforia.

Señales de tracción real (Gartner, "IA 2026: 5 tendencias clave"):

- El **auge de la IA agéntica**: sistemas que planifican, razonan y ejecutan tareas de varios pasos de forma autónoma.
- **40%** de las aplicaciones empresariales incorporarán agentes de IA para tareas específicas en 2026 (Gartner).
- **62%** de las organizaciones ya experimentan con agentes de IA, y un **23%** ya los está escalando en alguna función (McKinsey, 2025).
- De los **LLM** (*Large Language Model, modelo de lenguaje de gran escala*) a "modelos del mundo": mayor eficiencia de aprendizaje y mejor planificación al simular escenarios antes de actuar.

Lectura del instructor: estas dos señales no se contradicen — son la misma curva de adopción. La mayoría de los pilotos falla porque se construyen sin arquitectura (el mega-prompt de la Clase 7), mientras la adopción real crece donde sí hay diseño deliberado de componentes, autonomía y control de riesgo. Esta sesión enseña exactamente esa arquitectura.

Marco de referencia bibliográfico y de mercado citado en la apertura:

Fuente	Tipo	Aporte
Chip Huyen — <i>AI Engineering: Building Applications with Foundation Models</i> (O'Reilly)	Libro	Ingeniería práctica de sistemas con Foundation Models (<i>modelos fundacionales, modelos base entrenados a gran escala y adaptables a múltiples tareas</i>)
Pascal Bornet et al. — <i>Agentic Artificial Intelligence</i>	Libro	Marco de negocio para IA agéntica
MIT (Massachusetts Institute of Technology), Stanford HAI (<i>Human-Centered Artificial Intelligence</i>)	Academia	Investigación de frontera y reportes de adopción
Gartner, McKinsey & Company	Consultoría	Cuantificación de adopción empresarial y frameworks de madurez
FAANG (<i>Facebook/Meta, Amazon, Apple, Netflix, Google</i> — <i>abreviatura para las grandes tecnológicas</i>), OpenAI, Anthropic	Industria	Laboratorios que definen los estándares técnicos de agentes
Presidencia del Consejo de Ministros del Perú, EU AI Act (<i>European Union Artificial Intelligence Act, reglamento europeo de IA</i>), National AI Initiative Office (EE. UU.)	Gobierno/regulación	Marco normativo emergente para sistemas de IA

2. Objetivos y Agenda de la sesión

Objetivos declarados:

1. Definir de forma diferenciada las soluciones agénticas (qué es y qué no es un agente).

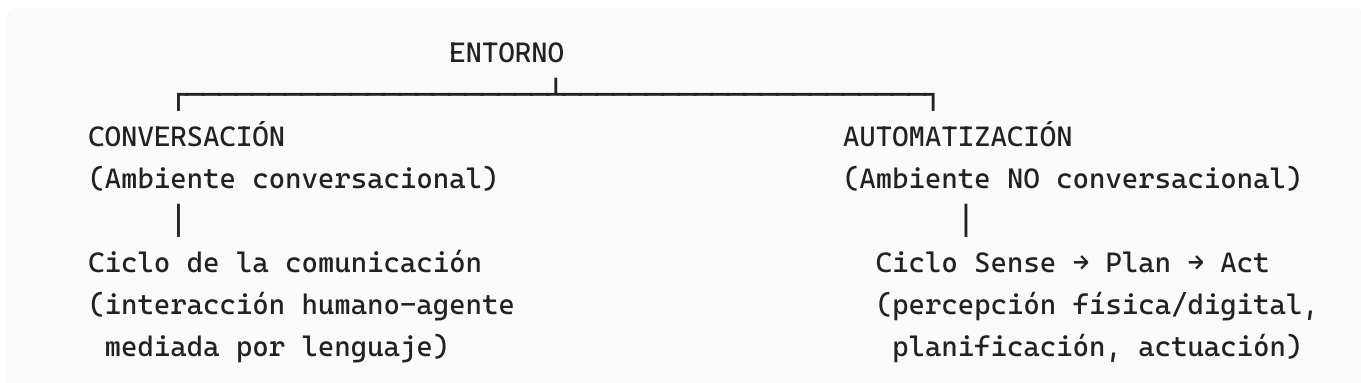
2. Plantear una arquitectura de solución agéntica (componentes + dimensiones funcionales).

Agenda completa (los dos bloques fusionados):

#	Tema	Bloque
1	Conversational Framework	A
2	Agent Definition	A
3	Arquitectura de un Agente → Arquitectura de Componentes	A
4	Arquitectura de un Agente → Dimensiones Funcionales	A
5	Lab: Agent Profile Card → System Prompt; borrador inicial en Draw.io	A
6	Agentic Systems Classification	B
7	Orquestación (determinista y no determinista)	B
8	Global Research Frameworks: Anthropic, OpenAI, Gartner, McKinsey	B
9	SDLC: referencia CRISP-DM	B
10	Lab: borrador en Draw.io + <i>Mocking Agents</i> (sin agentes de LangChain)	B

3. Marco Conversacional — Entorno, Ambiente y el ciclo de la comunicación

El material abre la arquitectura desde la pregunta más básica: ¿en qué **entorno** actúa un **sistema de IA**? Y separa el entorno en dos familias:



3.1 El ciclo de la comunicación (ambiente conversacional)

El diagrama del material reproduce el **modelo clásico de comunicación** (raíz en la teoría de la información de Shannon-Weaver, 1948, y el modelo de funciones del lenguaje de Jakobson, 1960), con seis elementos:

Elemento	Rol en un agente conversacional
Emisor	Usuario o agente que inicia el mensaje
Mensaje	El contenido comunicado
Código	El sistema de símbolos compartido (idioma, formato: texto, imagen, audio)
Canal	El medio de transmisión (chat, voz, videollamada)
Receptor	Quien interpreta el mensaje (el agente, o el usuario en la respuesta)
Retroalimentación (<i>feedback</i>)	La respuesta que cierra el ciclo
Contexto	El marco situacional que da sentido al mensaje (aparece dos veces en el diagrama del material — al inicio y al cierre del ciclo, subrayando que el contexto envuelve toda la interacción)

Por qué importa para arquitectura de agentes: cada uno de estos seis elementos se traduce directamente en una decisión de diseño de la Sección 8 (Arquitectura de Componentes): el *Código* y el *Canal* determinan la **Communication Layer**; el *Contexto* se convierte en la capa de **Context Definition** (dominio + objetivos).

3.2 El ciclo Sense-Plan-Act (ambiente no conversacional / automatización física)

Para sistemas no conversacionales (el ejemplo del material es un robot humanoide instrumentado con cámaras, sensores IMU (*Inertial Measurement Unit, unidad de medición inercial*), sonares y sensores de fuerza), el ciclo es:

1. **Sense** (percibir) — recolectar datos del entorno vía sensores.
2. **Plan** (planificar) — decidir la siguiente acción según el objetivo.
3. **Act** (actuar) — ejecutar la acción sobre el entorno físico.

(*Investigación complementaria*) Este ciclo **Sense-Plan-Act** es el patrón clásico de la robótica **deliberativa** (planificación explícita antes de actuar), contrapuesto históricamente a la arquitectura **reactiva/subsumption** de Rodney Brooks (1986), que actúa sin un modelo explícito del mundo. Los agentes LLM modernos con *tool calling* son, en esencia, una versión **deliberativa** de este ciclo, donde "Plan" es responsabilidad del razonamiento del modelo (ver también el patrón **ReAct**, §11.1).

3.3 Ejemplo trabajado: BurSee, el asesor bursátil

El material usa una "ID Robot Card" recurrente a lo largo de toda la sesión — un formato tipo ficha de identidad para especificar un agente:

ID ROBOT

Nombre: BurSee

Interfaz de comunicación: Conversación (diálogo)

Canal: Chat, Voz (teléfono, notas), videollamada

Código (modalidad): español, español + imagen, español + sonidos

Contexto: Banca y Finanzas

Dominio: Asesoría Bursátil

Esta ficha es la semilla del **Agent Profile Card** que se pide construir en el laboratorio (§9).

4. ¿Qué es (y qué NO es) un Agente de IA? — Definición de Gartner

Definición citada (Gartner):

"Los agentes de IA son entidades de software autónomas o semi-autónomas que usan técnicas de IA para percibir, tomar decisiones, tomar acciones y lograr objetivos en su ambiente digital o físico."

Explícitamente, el material lista lo que NO es un Agente de IA:

No es un agente	Por qué
Un LLM por sí solo	Es un generador de texto; no percibe ni actúa sobre un entorno por iniciativa propia
Instrucciones de tareas específicas	Ejecutan un guion fijo, sin decisión propia
Funciones de software automatizadas	Automatización determinista sin razonamiento
Workflows RPA (<i>Robotic Process Automation, automatización robótica de procesos</i>)	Reglas fijas sobre una interfaz; no hay planificación dinámica
Asistentes conversacionales	Responden, pero no necesariamente actúan de forma autónoma sobre un entorno
Una interfaz a un asistente	Es una capa de presentación, no un agente

El criterio distintivo, repetido de tres formas distintas en la sesión: percepción + decisión autónoma + acción + objetivo, en un entorno (digital o físico). Sin **las cuatro** a la vez, no es un agente — es alguno de los ítems de la lista anterior.

4.1 Investigación complementaria — la definición formal de agente racional

El material no formaliza matemáticamente la definición de Gartner, pero es útil anclarla a la definición clásica de **agente racional** de Russell & Norvig (*Artificial Intelligence: A Modern Approach*), que es la raíz académica de todo lo que se enseña en esta sesión.

Un agente se define como una función que mapea una **secuencia de percepciones** a una **acción**:

$$f : P^* \rightarrow A$$

donde P^* es el conjunto de todas las secuencias finitas de percepciones posibles, y A es el conjunto de acciones disponibles. Un **agente racional** es aquel que, en cada instante, elige la acción que maximiza su desempeño esperado dado lo que ha percibido hasta ese momento:

$$a^* = \arg \max_{a \in A} \mathbb{E} \left[U(s') \mid s, a \right]$$

donde s es el estado actual (interno + del entorno), s' es el estado resultante tras ejecutar a , y $U(\cdot)$ es la función de utilidad que codifica el objetivo del agente. Esta fórmula es, en esencia, la versión matemática de "lograr objetivos en su ambiente" de la definición de Gartner: el agente no solo actúa, actúa **para maximizar** algo definido por su objetivo.

El marco PEAS (*Performance, Environment, Actuators, Sensors* — desempeño, entorno, actuadores, sensores) de Russell & Norvig es, de hecho, casi un calco de las **Dimensiones Funcionales** que el material introduce en la §6: Objetivo (Performance), Entorno/Contexto (Environment), Acciones (Actuators) y Conocimiento/Memoria (Sensors, en sentido amplio de entrada de información).

5. La taxonomía de la Automatización Conversacional — la matriz de Boris Alzamora

Esta es la tabla más densa del material (páginas 15-16), construida por el propio instructor como marco propio ("By: Boris Alzamora, AI Solution Architect"). Cruza dos paradigmas de generación de respuesta con cinco dimensiones funcionales.

5.1 Los dos paradigmas de columna

	Retrieval Based	Generative Based
Definición	Respuestas preestablecidas: base de datos, XML (<i>eXtensible Markup Language</i>), TXT (<i>texto plano</i>)	Texto generado token por token, prediciendo el siguiente token
Subtipos (3 columnas cada uno en el original)	Option/Quick Reply/Keywords · Single Utterance · Context+Utterance	Input Prompt · Input Prompt+Context · System Prompt+Context+Input Prompt

5.2 Las cinco dimensiones (filas)

Dimensión	Retrieval Based	Generative Based
Input	Opción/Quick Reply/Keywords → Utterance única → Contexto+Utterance	Prompt de entrada → +Contexto → System Prompt+Contexto+Prompt
Inference	Reglas: IVR (<i>Interactive Voice Response, respuesta de voz interactiva</i>), Prolog, motores de reglas → basado en intención (ML/DL : <i>Machine Learning/Deep Learning</i> , aprendizaje automático/profundo — Dialogflow, LUIS)	Generativo: SML (<i>Small Language Model, modelo de lenguaje pequeño</i>), LLM, GPT (<i>Generative Pre-trained Transformer</i>) — aprendido por predicción de tokens
Reply	Textos predefinidos, recuperados de base de datos	Tokens generados; texto generado a partir de aprendizaje previo (dominio abierto o específico)
Action	Estado de la acción mapeado a una opción o conjunto de reglas	Estado de la acción según su autonomía ; descripción textual de herramientas y cómo usarlas (⚠ la primera columna generativa —solo Input Prompt, sin contexto ni tools— se marca en rojo como " <i>Risky, not Safe</i> ")
Memory	Solo datos estructurados, largo plazo	Datos estructurados y no estructurados, posiblemente largo y corto plazo (mayormente corto plazo si no hay arquitectura de memoria explícita)

5.3 El punto de inflexión: de "Knowledge Expert" a "Agent"

El material resalta con un recuadro rojo las tres columnas generativas y las etiqueta con una línea de tiempo:

Se apoyan en Knowledge Bases	→	"Knowledge Expert"	(2024)
Se agencian de Herramientas	→	"Agent"	(finales de 2024)

Esto es la tesis implícita de toda la matriz: un sistema generativo que solo tiene contexto documental (RAG puro, *Retrieval-Augmented Generation*, generación aumentada por recuperación) es un "experto en conocimiento", NO un agente. Se convierte en agente cuando además tiene **herramientas** que puede invocar para actuar sobre su entorno — exactamente la cuarta condición de la definición de Gartner (§4).

6. Dimensiones Funcionales de un Agente

Retomando la ficha de BurSee, el material construye seis (y luego ocho) dimensiones funcionales que cualquier agente debe especificar:

Dimensión	Definición	Ejemplo (BurSee)
Objetivo	El propósito que el agente persigue	Brindar asesoría bursátil sobre el portafolio de usuarios
Conocimiento	Las fuentes de verdad que el agente consulta	Referencia a portafolios líderes y documentación de estrategia bursátil
Acciones	Lo que el agente puede ejecutar sobre su entorno	Compra/venta de acciones, obtener listado de activos, dar <i>insights</i>
Memoria	Qué recuerda entre interacciones	Transacciones pasadas (exitosas y fallidas); preferencias de activos y frecuencia de operaciones
Autonomía	Cuánto control tiene sobre sus propias acciones	Semi-autónomo: solicita confirmación del usuario antes de operar
Criticidad <i>(agregada en la 2ª pasada del material, p. 19)</i>	Nivel de riesgo relativo de su actuar, cuantificado	—
Guardrails	Filtrado de contenido dañino: violento, político, sexual, autodaño, otros	—

Dimensión	Definición	Ejemplo (BurSee)
Controles	Anonimización de datos personales, prevención de persistencia de datos sensibles	—

6.1 Las preguntas que motivan Criticidad/Guardrails/Controles

El material formula, sin resolverlas técnicamente en esta sesión, cuatro preguntas que son el puente hacia **Risk Management** (gestión de riesgo):

¿Y si alucina? ¿Y si se equivoca de usuario? ¿Y si me hace perder dinero? ¿Si diagnostica mal a un paciente?

Risk Management → **Guardrails & Controls** es la respuesta explícita del material a estas cuatro preguntas — y es, en esencia, el mismo par conceptual (contrato + guardrail = doble red) que cerró la Clase 7 del Módulo 3, ahora aplicado a nivel de arquitectura completa de agente, no solo a un paso de un pipeline.

6.2 Investigación complementaria — cuantificar la criticidad

El material introduce "Criticidad" como *"nivel de riesgo en su actuar, cuantificación de riesgo relativa"*, sin dar una fórmula. Un enfoque estándar de gestión de riesgo (usado en ciberseguridad y en *risk management* de sistemas de IA) es expresar el riesgo esperado como:

$$R = P(\text{fallo}) \times I(\text{impacto})$$

donde $P(\text{fallo})$ es la probabilidad estimada de que la acción del agente falle o alucine, e $I(\text{impacto})$ es el costo (financiero, reputacional, de seguridad) si eso ocurre. Esta es la lógica exacta detrás del *tool safeguard rating* (bajo/medio/alto) que la Clase 7 ya había introducido para las herramientas de un agente, y que reaparece aquí como la dimensión de **Criticidad**: una acción de bajo $P(\text{fallo})$ pero altísimo $I(\text{impacto})$ (p. ej. una transferencia bancaria) sigue siendo de riesgo alto, y por tanto exige el mismo nivel de guardrail/HITL que una acción frecuente pero de bajo impacto.

6.3 Gartner — "Mind the AI Agency Gap"

El material inserta aquí un gráfico de Gartner (2024) que posiciona tres tipos de sistema en cinco ejes de "agencia" (capacidad de actuar con independencia):

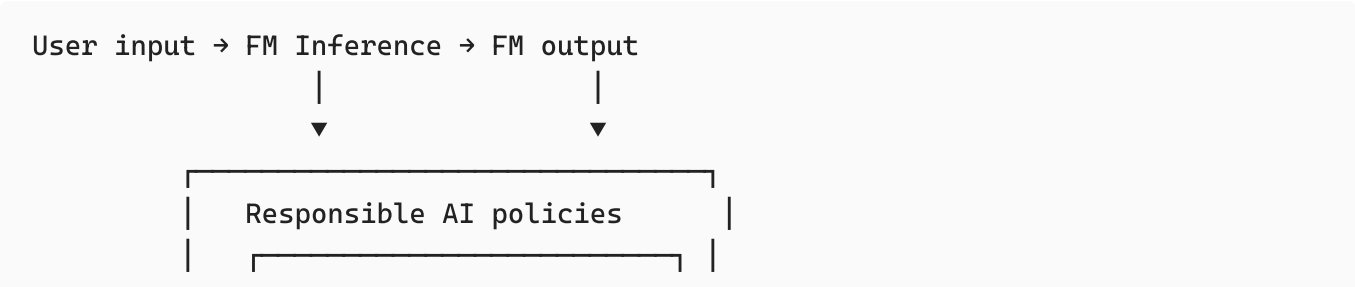
Eje (Baja agencia → Alta agencia)	Human agency	LLM-based assistants	Deterministic chatbots
Static → Adaptive	Alta	Media	Baja
Reactive → Proactive planning	Alta	Media	Baja
Simple tasks → Complex goals	Alta	Media	Baja
Simple environment → Complex environment	Alta	Media-baja	Baja
Supervised → Autonomous	Alta	Baja-media	Media (<i>cruce atípico: un chatbot determinista puede ser "no supervisado" porque simplemente no tiene margen de desviarse, no porque sea autónomo en el sentido pleno</i>)

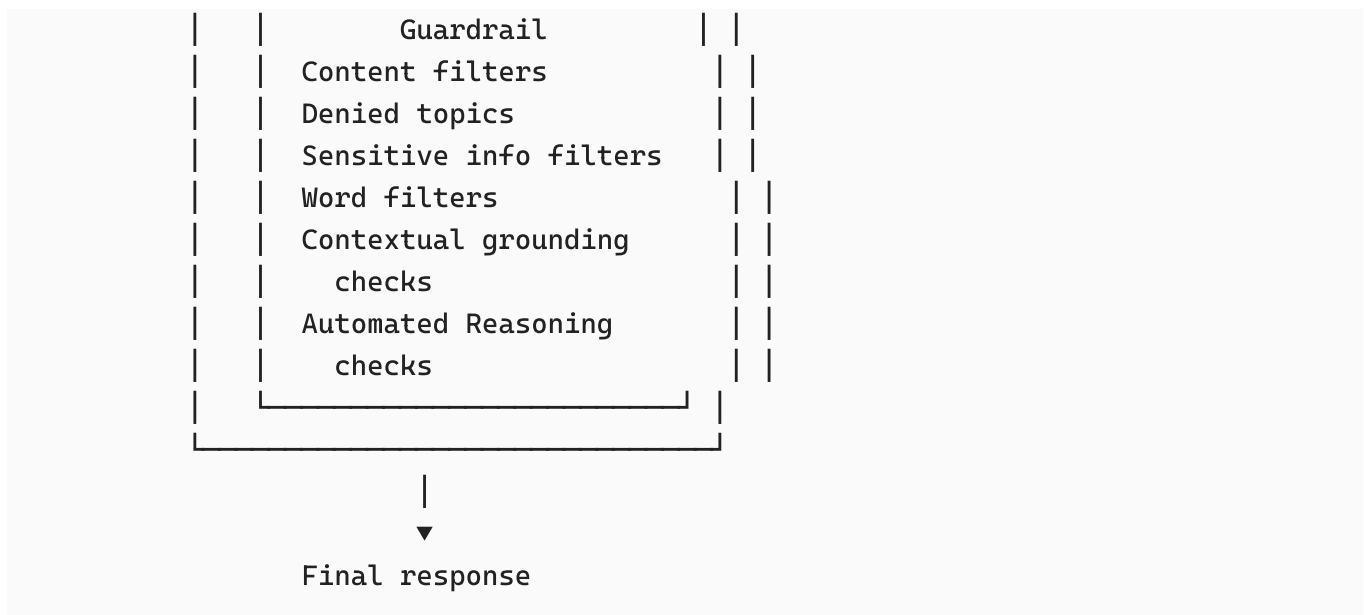
La brecha ("AI agency gap") es la distancia entre dónde están hoy los asistentes basados en LLM y la agencia humana plena. El mensaje implícito: los "agentes" de 2024-2025 todavía están, en la mayoría de los ejes, más cerca de los chatbots deterministas que de la agencia humana — de ahí la importancia de ser honesto sobre el nivel de autonomía real de lo que se construye (conecta directamente con el framework de niveles de la §11).

7. Guardrails y Controles — de la teoría a la implementación en la nube

El material documenta cómo los dos mayores proveedores de nube implementan la capa de guardrails a nivel de plataforma (no solo como diseño conceptual, como en la Clase 7, sino como producto):

7.1 AWS — arquitectura de Guardrails (Amazon Bedrock)





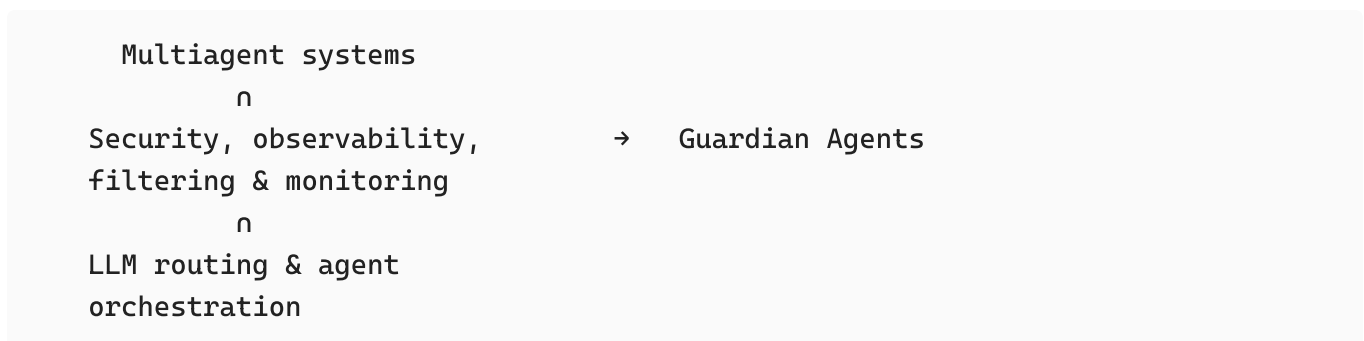
(**FM**: Foundation Model, *modelo fundacional*). Nótese que AWS aplica el guardrail **tanto al input del usuario como al output del modelo**, exactamente como la taxonomía de guardrails de OpenAI vista en la Clase 7 (entrada/salida). Lo nuevo aquí son los **Automated Reasoning checks**: verificaciones lógicas formales (no solo clasificadores estadísticos) sobre la coherencia de la salida — un guardrail de tipo "reglas" pero implementado con razonamiento simbólico, no solo *pattern matching*.

7.2 Microsoft Azure — AI Content Safety

La captura del material muestra el flujo de configuración de un filtro de contenido en **Azure AI Foundry**, con cuatro categorías estándar (violento, odio, sexual, autolesión), cada una con un **umbral de severidad ajustable** (Bajo/Medio/Alto) y una acción de "Annotate and block" (anotar y bloquear). Esto es la implementación literal, en interfaz de producto, del guardrail de **Moderación** de la taxonomía de OpenAI.

7.3 Gartner — "From Guardrails to Guardian Agents" (2025)

El material cierra esta sección con una tendencia de 2025: los guardrails estáticos evolucionan hacia **"Guardian Agents"** — agentes especializados cuya única función es vigilar a otros agentes.



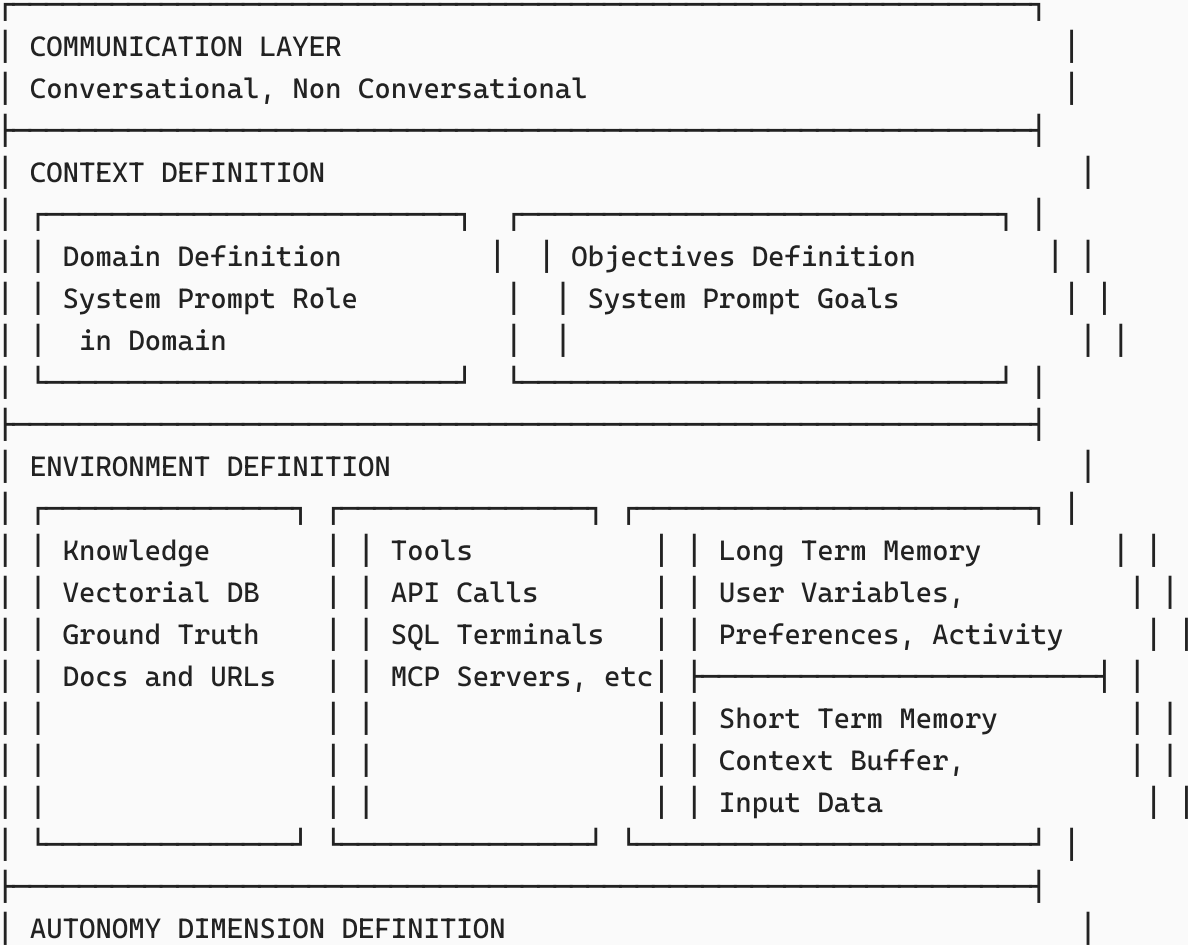
Puntos clave del material:

- El **abuso de agentes** ("Agent Abuse") aumentará a medida que se despliegan más agentes en producción.
- La respuesta: monitoreo de seguridad, enmascaramiento (*masking*) y filtrado de datos, más **logging de API** y supervisión que permitan auditar sistemas multiagente compuestos.
- Ejemplo citado: **aiXplain**.

Conexión con el resto de la sesión: un Guardian Agent no es más que un agente (con su propia definición de la §4) cuyo Objetivo es vigilar a otros agentes, y cuyas Acciones son de tipo auditoría/bloqueo — es la aplicación recursiva de la misma arquitectura de componentes que se define en la §8, aplicada a la meta-tarea de gobernanza de agentes.

8. Arquitectura de Componentes de un Agente — el diagrama integrador

Esta es la diapositiva más importante de la sesión (página 23), el diagrama propio del instructor que sintetiza **todo** lo anterior en una sola arquitectura de cinco capas apiladas:



Fully Autonomous, Semi Autonomous, Constrained	
CRITICALITY DIMENSION DEFINITION	
Guardrails, Evals, LLMaaJ and Controls	

(**MCP**: Model Context Protocol, *protocolo abierto de Anthropic para conectar modelos con herramientas y fuentes de datos externas de forma estandarizada*. **LLMaaJ**: LLM as a Judge, *uso de un modelo de lenguaje como evaluador automático de la calidad de otra salida*.)

8.1 Lectura capa por capa

Capa	Qué resuelve	Con qué concepto de la sesión se conecta
Communication Layer	Cómo entra/sale la información (§3)	Conversacional vs. No conversacional
Context Definition	Quién es el agente y para qué existe	El <i>System Prompt</i> codifica Rol (Dominio) + Objetivos — esto es, literalmente, el contrato de I/O de la Clase 5/7 aplicado al nivel de todo el agente
Environment Definition	Con qué "sabe", "hace" y "recuerda"	Conocimiento (RAG/bases vectoriales), Herramientas (<i>Tool calling</i>), Memoria de largo y corto plazo
Autonomy Dimension	Cuánto decide el agente sin supervisión	Escala Fully Autonomous / Semi Autonomous / Constrained — versión de tres niveles del framework de 5 niveles de la §11
Criticality Dimension	Cómo se controla el riesgo	Guardrails (§7), <i>Evals</i> (evaluaciones sistemáticas), LLMaaJ, Controles

8.2 Formalización — investigación complementaria

Aunque el material no lo presenta como notación formal, el diagrama se puede expresar como una **tupla de componentes**, útil para especificar un agente de forma no ambigua (y directamente traducible a un *System Prompt* estructurado, que es justamente el objetivo del Lab 1):

$$\mathcal{A} = \langle C, (D, O), (K, T, M_{LT}, M_{ST}), \alpha, \kappa \rangle$$

donde:

- *C* = capa de comunicación (canal, código, modalidad),
- *D* = definición de dominio, *O* = definición de objetivos,

- K = conocimiento (bases vectoriales, *ground truth*, documentos), T = herramientas (llamadas API, terminales **SQL** — *Structured Query Language*, lenguaje de consulta estructurado —, servidores MCP),
- M_{LT} = memoria de largo plazo, M_{ST} = memoria de corto plazo,
- $\alpha \in \{\text{Constrained, Semi-Autonomous, Fully Autonomous}\}$ = nivel de autonomía,
- κ = nivel de criticidad (con sus guardrails, *evals* y controles asociados).

Diseñar un agente, bajo este framework, es simplemente **instanciar cada componente de la tupla** — que es exactamente la secuencia de preguntas que el Lab 1 (§9) pide responder.

9. Lab 1 — De la *Agent Profile Card* al *System Prompt*

Instrucción del material:

Tomar como referencia sus proyectos y, para cada uno, definir un *Agent Profile Card* que luego se implemente en una definición de Agente (System Prompt). Definir Componentes (Herramientas, Conocimiento, Memoria, otros) y Dimensiones Funcionales (Autonomía, Criticidad de tareas).

9.1 Plantilla reconstruida de *Agent Profile Card* (a partir del ejemplo BurSee)

```
ID AGENTE
Nombre: [ej. BurSee]
Interfaz de comunicación: [Conversación / No conversacional]
Canal: [Chat, Voz, videollamada, API, ...]
Código (modalidad): [idioma(s) + imagen/audio si aplica]
Contexto (dominio de negocio): [ej. Banca y Finanzas]
Dominio (rol específico): [ej. Asesoría Bursátil]

Objetivo: [propósito medible del agente]
Conocimiento: [fuentes de verdad: docs, bases vectoriales, APIs de referencia]
Acciones: [lista de herramientas / operaciones que puede ejecutar]
Memoria:
  - Largo plazo: [preferencias, historial, variables de usuario]
  - Corto plazo: [buffer de contexto de la conversación actual]
Autonomía: [Constrained / Semi-Autonomous / Fully Autonomous + condición de escalamiento]
Criticidad: [nivel de riesgo relativo + justificación]
Guardrails: [categorías aplicables: violento, sexual, autodaño, político, otros]
```

Controles: [anonimización PII, prevención de persistencia de datos sensibles, otros]

9.2 De la ficha al *System Prompt*

El puente explícito del laboratorio es: cada campo de la ficha se traduce a una sección del *System Prompt* del agente. Es la misma lógica de "contrato I/O" de la Clase 5 del Módulo 3, ahora escrita como instrucción de sistema completa en vez de contrato de un solo paso:

Eres {Nombre}, un agente de {Dominio} que opera en el contexto de {Contexto}.

ROL Y OBJETIVO

{Objetivo}

CONOCIMIENTO DISPONIBLE

{Conocimiento – qué fuentes consultar y cómo citarlas}

HERRAMIENTAS DISPONIBLES

{Acciones – nombre, descripción, cuándo usarla, nivel de riesgo}

MEMORIA

{Qué debes recordar entre turnos y qué es efímero}

AUTONOMÍA

{Qué puedes hacer sin confirmación y qué requiere aprobación humana}

RESTRICCIONES Y GUARDRAILS

{Qué no debes hacer, qué debes filtrar, qué datos no debes exponer}

Nota práctica sobre el 2º laboratorio (Draw.io): el material pide, en paralelo, bocetar esta misma arquitectura como un **diagrama de componentes** (no solo texto) — es decir, dibujar las cinco capas de la §8 ya instanciadas con los datos concretos del proyecto propio, como primer artefacto de diseño antes de escribir código.

10. Clasificación de Sistemas Agénticos — Workflows, Agents, MultiAgents

10.1 Anthropic vs. OpenAI — dos definiciones que convergen

El material reproduce, en paralelo, las definiciones de los dos laboratorios líderes (ya introducidas parcialmente en la Clase 7 del Módulo 3, y ahora contrastadas directamente):

	OpenAI	Anthropic
Agente	"Sistemas que realizan tareas de forma autónoma en tu nombre."	"Sistemas donde los LLMs dirigen dinámicamente sus propios procesos y el uso de herramientas, manteniendo el control sobre cómo realizan las tareas."
Workflow	"Una secuencia de pasos determinada, orientada a resolver necesidades de un usuario."	"Sistemas donde los LLMs y las herramientas se orquestan mediante rutas de código predeterminadas."
Lo que NO es agente	"Las aplicaciones que integran un LLM pero no lo usan para controlar la ejecución del flujo de trabajo no son agentes."	(implícito: si el código decide el orden, es workflow, no agente)

Ambas definiciones coinciden en el mismo criterio distintivo que cerró la Clase 7: quién dirige el orden — el código (workflow) o el LLM en tiempo de ejecución (agente). Esta sesión no introduce un criterio nuevo; lo consolida citando ambas fuentes primarias lado a lado.

10.2 El framework propio del instructor — cuatro niveles de orquestación

Boris Alzamora propone una progresión de cuatro niveles, cruzando **qué se orquesta** (el plan vs. las herramientas):

Nivel	Definición	Orquestación del plan de resolución	Orquestación de herramientas y conocimiento
LLM Augmented Features	Data pipeline potenciado con invocación a LLMs	— (no aplica; no hay "plan")	—
Workflows	Paso a paso determinado y diseñado por el desarrollador	❌ Fija (código)	❌ Autonomía lograda solo por pasos potenciados con GenAI
Agents	Plan de resolución creado de forma autónoma	✅ Autónoma (el LLM decide el plan)	✅ Pasos ejecutados por asociación a herramientas, conocimiento y memoria propias

Nivel	Definición	Orquestación del plan de resolución	Orquestación de herramientas y conocimiento
MultiAgents	Plan de resolución creado de forma autónoma, ejecutado por una red de agentes	✓ Autónoma	✓ Pasos ejecutados por asociación a otros agentes , y estos a su vez a herramientas, conocimiento y memoria

Las dos orquestaciones marcadas en el diagrama original con colores distintos:

-  **Orquestación del Plan de Resolución** — abarca Workflows, Agents y MultiAgents (todos tienen *algún* plan, solo cambia quién lo define).
-  **Orquestación de Herramientas y Conocimiento** — abarca Agents y MultiAgents (los Workflows no "asocian" herramientas dinámicamente; las invocan en el orden fijo del código).

Esta tabla es, en la práctica, la respuesta operativa a "¿cuándo necesito un MultiAgente y no un solo Agente?": cuando el plan de resolución requiere que **distintos** conjuntos de herramientas/conocimiento se activen bajo la coordinación de agentes especializados, en vez de un único agente que intenta abarcar todas las herramientas él mismo.

11. Frameworks de progresión de autonomía

11.1 AI Agent Progression Framework (Rakesh Gohel)

Un framework de cinco niveles, citado del investigador Rakesh Gohel, que cuantifica la relación entre autonomía y esfuerzo de implementación:

Nivel	Nombre	Características
1	Rule-based Automation	Sistemas rígidos, guiados por reglas, sin aprendizaje. Lógica <i>if-then</i> simple; acciones manuales con herramientas
2	Intelligent Automation	Sistemas de IA básicos con autonomía limitada. ML simple para reconocimiento de patrones; algo de soporte a la decisión
3	Agentic Workflows	Agentes que razonan y aprenden de retroalimentación. Comprensión de lenguaje natural; orquestación de herramientas

Nivel	Nombre	Características
4	Semi-Autonomous Agents	Agentes dirigidos por objetivos, con conciencia multimodal. Perciben entornos complejos; planifican usando experiencia pasada
5	Fully Autonomous Agents	Agentes auto-mejorables con autonomía total. Aprendizaje y razonamiento continuos; sin intervención humana requerida

Regla del framework: *"Mientras más subes en el framework, menos: tiempo necesitas para implementación, instrucciones que debes dar, control que tienes."* Es un *trade-off* explícito — más autonomía significa, estructuralmente, **menos control directo** del diseñador. Esta es la misma tensión que ya apareció en la Clase 7 ("el agente paga su flexibilidad en latencia y coste") pero formulada ahora como pérdida de control, no solo de eficiencia.

11.2 Gartner — "Agentic AI vs. AI Agents": un continuo, no una dicotomía

Gartner enfatiza que **"Agentic AI" es un continuo**, no una categoría binaria:



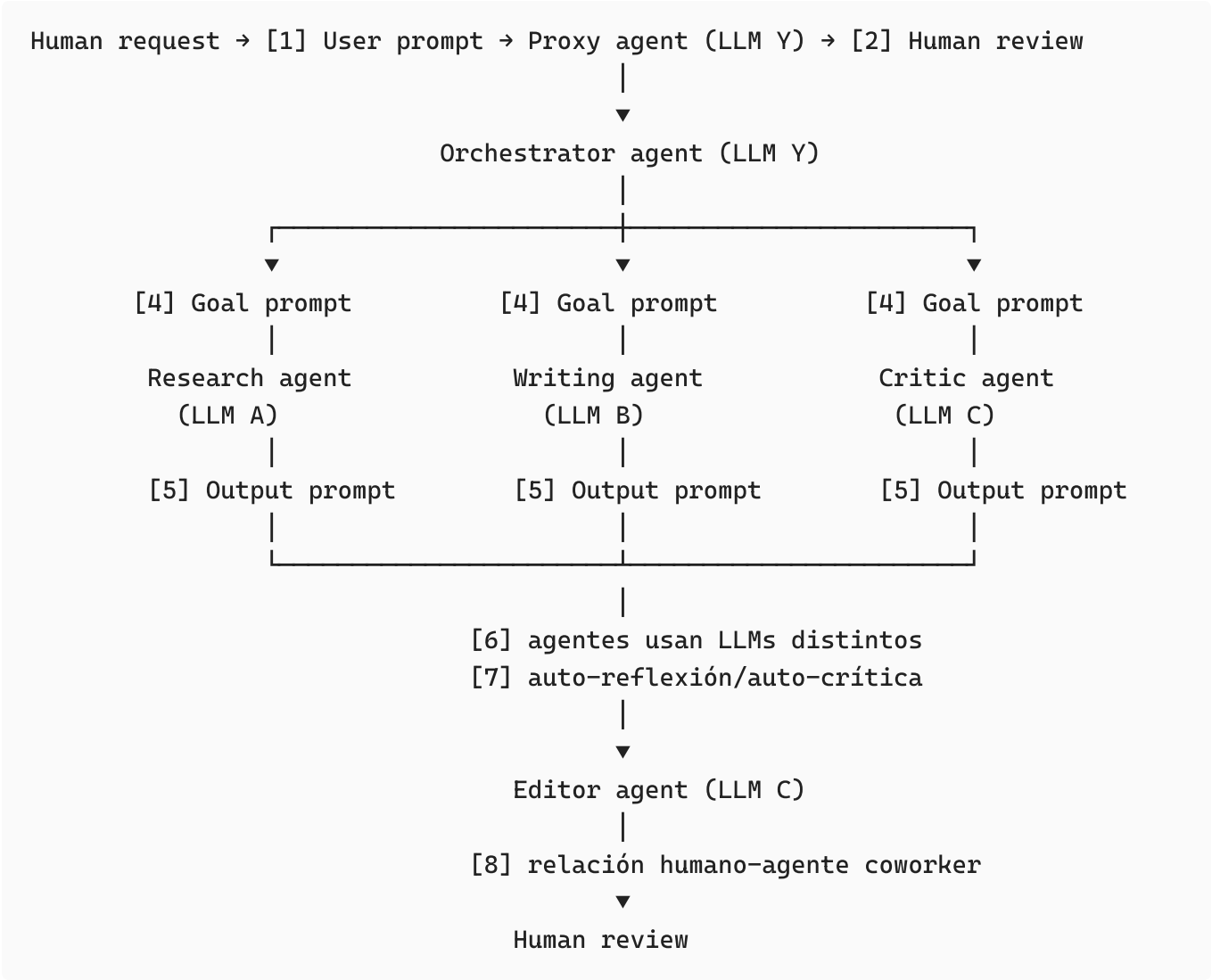
Matiz importante citado textualmente: *"AI assistants are at the lower end, but not all assistants qualify as agentic AI."* — no todo asistente conversacional es, por defecto, "agéntico"; solo lo es si se ubica en algún punto significativo de este continuo de capacidades (planificación, independencia, evolución).

11.3 Investigación complementaria — el paralelo con los niveles de conducción autónoma

Aunque el material no hace esta analogía, vale la pena nombrarla porque clarifica mucho el framework de 5 niveles de Rakesh Gohel: es estructuralmente idéntico a los **niveles SAE de automatización vehicular** (*Society of Automotive Engineers*, 0 a 5), donde el Nivel 0 es control humano total y el Nivel 5 es autonomía completa sin intervención. La analogía es útil porque en automoción esos niveles vienen acompañados de un principio muy relevante para IA: **subir de nivel no es gratuito** — cada salto de nivel exige mecanismos de seguridad, *fallback* y supervisión proporcionalmente más sofisticados, exactamente el mismo principio detrás de la dimensión de **Criticidad** (§6) de esta sesión.

12. Orquestación Multiagente en la práctica — el caso "Blog Writer" de Gartner

El material analiza un diagrama de referencia de Gartner: un **MAS** (*Multi-Agent System*, sistema multiagente) que escribe un blog post con ocho pasos numerados.



Los ocho pasos anotados en el original:

#	Paso
1	El usuario pide escribir un blog
2	Se comunica con el humano
3	Coordina las capacidades de otros agentes
4	El sub-objetivo es un prompt para el siguiente agente
5	Los sub-agentes devuelven su output

#	Paso
6	Los agentes de un MAS pueden usar LLMs distintos
7	Auto-reflexión o auto-crítica
8	Relación de coworker humano + agente de IA

12.1 Las cuatro preguntas de evaluación del material

El material acompaña este diagrama con cuatro preguntas diseñadas como rúbrica de análisis para *cualquier* MAS que se encuentre en la práctica:

- 1. **¿Este MAS es autónomo?** (¿decide su propio plan, o sigue un guion?)
- 2. **¿Este MAS tiene decisiones críticas?** (¿hay acciones de alto riesgo/impacto en la cadena?)
- 3. **¿Tiene un paso a paso definido?** (¿es workflow con LLMs, o el orquestador decide dinámicamente?)
- 4. **¿Usan el mismo LLM?** (¿es una única llamada reutilizada, o distintos modelos especializados por rol?)

Aplicadas al caso Blog Writer: (1) parcialmente autónomo — el *Proxy agent* y el *Orchestrator* dirigen dinámicamente, pero hay *human review* en dos puntos; (2) criticidad baja-media — es contenido, no una transacción financiera; (3) no tiene un paso a paso 100% fijo — el orquestador coordina dinámicamente a los sub-agentes; (4) **no** — el diagrama marca explícitamente que Research (LLM A), Writing (LLM B) y Critic/Editor (LLM C) usan modelos distintos, una decisión de diseño deliberada para especializar cada rol.

13. Ecosistema de Frameworks de Orquestación — investigación complementaria

El material solo muestra los logotipos de ocho frameworks (página 31), sin describirlos. Dado que el propio material remarca **LangGraph** y **LangChain** con un recuadro (probablemente por ser los que se usarán en el laboratorio de código, tal como en la Clase 7), aquí se completa la tabla comparativa que el PDF no desarrolla:

Framework	Nombre completo / origen	Paradigma de orquestación	Cuándo se usa típicamente
LangChain	LangChain (Harrison)	Cadenas (<i>chains</i>) de pasos encadenados, con	Prototipado rápido de pipelines RAG y agentes

Framework	Nombre completo / origen	Paradigma de orquestación	Cuándo se usa típicamente
	Chase, código abierto)	soporte de agentes vía <i>tool calling</i>	simples
LangGraph	LangGraph (extensión de LangChain)	Grafos de estados explícitos — cada nodo es un paso/agente, las aristas son transiciones condicionales	Agentes con lógica de control compleja, ciclos, y necesidad de <i>checkpoints</i> /HITL explícitos en el grafo
CrewAI	CrewAI (código abierto)	Basado en roles : se definen "agentes" con un rol, un objetivo y herramientas, organizados en un " <i>crew</i> " (equipo)	Sistemas multiagente con división de trabajo tipo equipo humano (investigador, redactor, editor)
AutoGen	Microsoft AutoGen	Conversación multiagente: los agentes se comunican entre sí en un chat grupal simulado	Investigación y prototipado de patrones de conversación entre agentes
Semantic Kernel	Microsoft Semantic Kernel	SDK (<i>Software Development Kit</i>) de integración empresarial: <i>planners</i> , <i>plugins</i> (equivalentes a <i>tools</i>), memoria	Integración de agentes dentro de aplicaciones .NET /enterprise ya existentes
AutoGPT	Auto-GPT (código abierto)	Agente autónomo de bucle continuo: se autoasigna sub-tareas para lograr un objetivo de alto nivel	Demostraciones de autonomía total (Nivel 5 de la §11.1); poco usado en producción por falta de control
smolagents	smolagents (Hugging Face)	Agentes minimalistas centrados en generación de código como acción (en vez de JSON de <i>tool calls</i>)	Casos donde escribir/ejecutar código Python es la herramienta principal del agente
Strands Agents	Strands Agents (AWS , Amazon Web Services)	<i>Model-driven</i> : define agentes con un modelo + herramientas + <i>prompt</i> , con orquestación nativa hacia servicios AWS	Agentes desplegados dentro del ecosistema AWS/Bedrock

Nota de consistencia con la Clase 7: el material de esta sesión pide explícitamente *"Mocking Agents in VSCODE (NO LANGCHAIN AGENTS)"* en la primera versión de la agenda (p. 11) y luego, en la versión final del laboratorio (p. 34), lo actualiza a *"Mocking Agents in Claude"*. Esto confirma la misma filosofía de la Clase 7: **la vía principal del curso es simular la arquitectura (contratos, gates, roles, herramientas) manualmente o con un asistente de código como Claude, sin depender de un framework de agentes** — los frameworks (LangGraph, CrewAI, etc.) son la "vía pro" opcional, no el requisito.

14. Arquitecturas de referencia en la nube — AWS y Azure

14.1 AWS — orquestación de agentes basada en grafos

El diagrama de AWS (p. 32) muestra un patrón de **orquestación basada en grafos** que combina tres categorías de agente:

Categoría	Ejemplo en el diagrama	Rol
Native agent	Bedrock Agent	Agente nativo del proveedor de nube
Open source agents	Search Agent, LangChain Agent, RAG Agent	Agentes construidos con frameworks abiertos
Proprietary agent	CrewAI Agent	Framework propietario/comercial

El flujo: User query → Query re-write → Graph-based agent orchestration → [ejecución en paralelo de los distintos agentes] → Grader Agents (evalúan si hay Match) → Human-in-the-loop → Answers . Si no hay *match*, el flujo puede derivar hacia generación de imagen (*Text-2-image*) según el tipo de resultado esperado.

El componente clave a resaltar: los **Grader Agents** — agentes cuya única función es evaluar la calidad de las respuestas de los otros agentes antes de que lleguen al humano. Es una instancia concreta de **LLMaaJ** (LLM as a Judge, ya mencionado en la §8) y del mismo patrón **Evaluator-Optimizer** que la Clase 7 del Módulo 3 había citado sin desarrollar.

14.2 Azure — árbol de decisión de arquitectura para Apps/Agentes de IA

El diagrama de Azure AI Foundry (p. 33) es un extenso árbol de decisión que ayuda a elegir componentes según tres preguntas de arranque:

1. ¿Quieres un enfoque tipo asistente ("wizard") para crear rápido una app de IA fundamentada en tus datos? → AI Foundry con datos propios.
2. ¿Quieres integrar el LLM en tu aplicación existente, o necesitas control y flexibilidad totales de tu copiloto? → define *Orchestration Runtime and Frontend* (microservicios, web apps, contenedores, *serverless*, telefonía/mensajería).
3. A partir de ahí, el árbol se ramifica en cuatro grandes bloques de decisión: **Tools** (Plugins/Workflows, AI Services, Code Interpreter), **Memory** (bases vectoriales, historial de chat, *knowledge graphs*), **Reasoning Engine** (modelo — multimodal, *frontier models*, *embeddings*, razonamiento avanzado tipo "System 2") y **Quality Attributes** (AI Safety, Evals & LLMOps, Security, escalabilidad/confiabilidad).

Valor pedagógico de este diagrama: es la contraparte de *implementación en la nube* de la Arquitectura de Componentes de la §8 — Tools = *Environment Definition* → Tools; Memory = *Environment Definition* → Long/Short Term Memory; Reasoning Engine = el modelo que ejecuta *Context Definition*; Quality Attributes = *Criticality Dimension*. La misma arquitectura conceptual, expresada ahora como catálogo de servicios concretos de un proveedor de nube.

15. SDLC de un Agente — referencia CRISP-DM (investigación complementaria)

La agenda del material (p. 11) nombra explícitamente "**SDLC: CRISP-DM Reference**" como tema, pero las diapositivas disponibles no lo desarrollan en detalle — probablemente por tiempo, o por tratarse de contenido cubierto verbalmente en la sesión en vivo. Dado que es un framework estándar y de alto valor para estructurar el ciclo de vida de un proyecto de agentes, se documenta aquí su aplicación:

CRISP-DM (*Cross-Industry Standard Process for Data Mining*) es un proceso de seis fases, iterativo, originalmente diseñado para proyectos de minería de datos/ciencia de datos — y directamente adaptable al desarrollo de agentes de IA porque comparte la misma naturaleza experimental (no se puede especificar un agente completo por adelantado; se refina con evidencia):

Fase CRISP-DM	Aplicación a un proyecto de Agente
1. Business Understanding (comprensión del negocio)	Definir el Objetivo del agente (§6) y el caso de negocio: ¿qué problema resuelve, para quién, con qué métrica de éxito?
2. Data Understanding (comprensión de los datos)	Explorar las fuentes de Conocimiento disponibles (§8: <i>Knowledge</i> — documentos, bases vectoriales, <i>ground truth</i>)

Fase CRISP-DM	Aplicación a un proyecto de Agente
	y su calidad
3. Data Preparation (preparación de los datos)	Construir la base de conocimiento (<i>chunking</i> , <i>embeddings</i> , indexación) y definir el esquema de Memoria (largo/corto plazo)
4. Modeling (modelado)	Diseñar la Arquitectura de Componentes completa (§8): elegir el modelo, definir herramientas, nivel de Autonomía, y — si aplica — la topología multiagente (§10-12)
5. Evaluation (evaluación)	Definir <i>Evals</i> y LLMAaJ (§8, Criticality Dimension): ¿el agente cumple el Objetivo con el nivel de Criticidad aceptable? Aquí se prueban también los Guardrails
6. Deployment (despliegue)	Poner el agente en producción con observabilidad (Clase 7, Concepto 6) y los controles de Guardian Agents si aplica (§7.3)

La naturaleza cíclica de CRISP-DM es, quizás, su aporte más importante para agentes: las flechas de retorno entre fases (de *Evaluation* de vuelta a *Business Understanding*, por ejemplo) formalizan lo que ya el Concepto 6 de la Clase 7 llamaba el "bucle de mejora continua" (*mido* → *encuentro el paso débil* → *mejoro* → *vuelvo a medir*), ahora aplicado al ciclo de vida completo del agente, no solo a un paso del pipeline.

16. Lab 2 — Draw.io + *Mocking Agents*

Instrucción del material (dos versiones, reflejando la evolución del curso):

- *Versión inicial de la agenda (p. 11):* borrador inicial en Draw.io + *Mocking Agents in VSCODE (NO LANGCHAIN AGENTS)*.
- *Versión final del laboratorio (p. 34):* borrador inicial en Draw.io + *Mocking Agents in Claude*.

Qué significa "*mocking* de agentes": simular el comportamiento de un sistema multiagente **sin implementar un framework real de agentes** (ni LangChain, ni CrewAI) — en su lugar, se simula manualmente cada "agente" como un *prompt* independiente (exactamente como el pipeline manual de la Clase 7: "*corres cada paso en el chat y copias la salida como entrada del siguiente*"), pero ahora con la topología de un sistema multiagente (Proxy → Orchestrator → Workers → Editor, como en el caso Blog Writer de la §12) en vez de un pipeline lineal de 3-4 pasos.

Por qué esta secuencia pedagógica es deliberada: obliga a internalizar la **arquitectura** (quién es cada agente, qué herramientas tiene, cómo se comunican, dónde está el HITL) antes

de delegarla a un framework que oculta esas decisiones detrás de abstracciones de código. Es la misma filosofía repetida en cada sesión del curso: la estructura conceptual es independiente de la herramienta de implementación.

17. Síntesis — lo que hay que llevarse de esta sesión

1. **Un agente de IA no es un LLM, ni un asistente conversacional, ni un workflow RPA:** es una entidad que percibe, decide, actúa y persigue un objetivo en su entorno — las cuatro condiciones a la vez (Gartner).
 2. **La matriz Retrieval vs. Generative** muestra que ni siquiera todo sistema generativo es un agente: un sistema generativo apoyado solo en bases de conocimiento es un "Knowledge Expert"; se vuelve "Agent" cuando además se agencia de herramientas para actuar.
 3. **Las Dimensiones Funcionales** (Objetivo, Conocimiento, Acciones, Memoria, Autonomía, Criticidad, Guardrails, Controles) son el checklist mínimo para especificar cualquier agente — y se traducen directamente en un *Agent Profile Card* y luego en un *System Prompt*.
 4. **La Arquitectura de Componentes de cinco capas** (Comunicación → Contexto → Entorno → Autonomía → Criticidad) es el diagrama de referencia de toda la sesión: cada decisión de diseño de un agente encaja en exactamente una de esas cinco capas.
 5. **Workflow vs. Agent vs. MultiAgent se distingue por quién orquesta el plan y quién orquesta las herramientas** — no por la tecnología usada. Anthropic y OpenAI convergen en el mismo criterio (¿código o LLM dirige el orden?); el framework propio del instructor añade el eje de "orquestación de herramientas/conocimiento" para distinguir Agent de MultiAgent.
 6. **La autonomía es un continuo, no una etiqueta binaria** (Gartner) — y subir de nivel (framework de 5 niveles de Rakesh Gohel) siempre cuesta control directo del diseñador, exactamente como en los niveles de conducción autónoma.
 7. **Los guardrails ya no son solo reglas estáticas:** la tendencia 2025 (Gartner) es hacia *Guardian Agents* — agentes especializados que vigilan a otros agentes en sistemas multiagente, con seguridad, observabilidad y enrutamiento como funciones nativas.
 8. **El desarrollo de un agente es un ciclo, no un proyecto lineal** (CRISP-DM adaptado): comprensión de negocio → comprensión de datos/conocimiento → preparación → modelado de la arquitectura → evaluación (*Evals/LLMaaS*) → despliegue, con retroalimentación continua entre fases.
 9. **La vía pedagógica del curso sigue siendo simular antes de automatizar:** *mocking* manual de agentes (en chat o con Claude) antes de adoptar un framework de orquestación (LangGraph, CrewAI, AutoGen, etc.) — la estructura importa más que la herramienta.
-

18. Checklist práctico — diseñar tu propio Agent Profile Card

Definición y encuadre:

- ☐ ¿Mi sistema percibe, decide, actúa y persigue un objetivo en un entorno — las cuatro condiciones a la vez? Si falta alguna, probablemente no es un agente (es un asistente, un RAG, o un workflow).
- ☐ ¿Mi sistema tiene herramientas para actuar sobre su entorno, o solo conocimiento para responder? Si es solo lo segundo, es un "Knowledge Expert", no un agente.

Comunicación y contexto:

- ☐ ¿Definé el canal, el código/modalidad y si el agente es conversacional o no conversacional?
- ☐ ¿El *System Prompt* especifica claramente el Dominio (rol) y los Objetivos del agente?

Entorno:

- ☐ ¿Especifiqué las fuentes de Conocimiento (bases vectoriales, *ground truth*, documentos) y cómo se actualizan?
- ☐ ¿Listé las Herramientas disponibles con su nivel de riesgo (bajo/medio/alto), tal como en los *tool safeguards* de la Clase 7?
- ☐ ¿Distinguí qué va en Memoria de Largo Plazo (preferencias, historial) vs. Corto Plazo (buffer de la conversación actual)?

Autonomía y criticidad:

- ☐ ¿Definé el nivel de autonomía (Constrained / Semi-Autonomous / Fully Autonomous) y qué acciones requieren confirmación humana?
- ☐ ¿Cuantifiqué la criticidad como $R = P(\text{fallo}) \times I(\text{impacto})$, o al menos la clasifiqué relativa a otras acciones del mismo agente?
- ☐ ¿Definé guardrails de al menos dos categorías (seguridad + PII, como en la Clase 7) y controles de anonimización?

Clasificación arquitectónica:

- ☐ ¿Es realmente un Agente (el LLM decide el plan) o me basta con un Workflow (plan fijo, código decide el orden)? Empieza por lo simple.
- ☐ Si necesito varios roles especializados con distintas herramientas/conocimiento, ¿considero un MultiAgente en vez de forzar un solo agente a hacer todo?
- ☐ Si construyo un MultiAgente: ¿definí quién es el Orchestrator, cuáles son los Workers, y dónde está el punto de *human review*?

Validación antes de escalar a un framework:

- ☐ ¿Ya *mockeé* (simulé manualmente, sin librería de agentes) el flujo completo al menos una vez, para confirmar que la arquitectura tiene sentido antes de programarla?
-

19. Quiz de la sesión (con respuestas)

#	Pregunta	Respuesta correcta
1	Según Gartner, ¿cuál de los siguientes SÍ califica como Agente de IA?	C — Un sistema que percibe su entorno, decide autónomamente y ejecuta acciones (compra/venta, consultas) para lograr un objetivo, con memoria de sus resultados pasados
2	En la matriz de Boris Alzamora, ¿qué distingue a un "Knowledge Expert" (2024) de un "Agent" (fin de 2024)?	B — El Agent se agencia de herramientas para actuar; el Knowledge Expert solo se apoya en bases de conocimiento
3	En la Arquitectura de Componentes de 5 capas, ¿en qué capa vive la definición de "Fully Autonomous / Semi Autonomous / Constrained"?	D — Autonomy Dimension Definition
4	¿Qué distingue a un Agent de un MultiAgent en el framework de 4 niveles del instructor?	A — En el MultiAgent, los pasos se ejecutan por asociación a <i>otros agentes</i> , que a su vez usan herramientas, conocimiento y memoria propias
5	Según el framework de Rakesh Gohel, ¿qué ocurre al subir de nivel de autonomía (1→5)?	B — Se necesita menos tiempo de implementación, menos instrucciones, pero también menos control directo del diseñador
6	¿Qué es un "Guardian Agent" según la tendencia 2025 de Gartner?	C — Un agente especializado en seguridad, observabilidad y monitoreo de otros agentes dentro de un sistema multiagente

20. Referencias

Del material original:

- Gartner — definición de Agentes de IA; "*Mind the AI Agency Gap*" (2024, CTMKT_3176213); "*Agentic AI vs. AI agents*" (2025, CTMKT_3848929); "*From Guardrails*

to Guardian Agents" (2025, CTMKT_3613687); "AI Agent Blog Writer Multiagent System" (817826_C).

- Anthropic — definición de Agentes vs. Workflows (consistente con *Building Effective Agents*, citado en la Clase 7 del Módulo 3).
- OpenAI — *A practical guide to building agents* (definición de Agente/Workflow, ya citada en la Clase 7).
- Huyen, Chip — *AI Engineering: Building Applications with Foundation Models*, O'Reilly.
- Bornet, Pascal et al. — *Agentic Artificial Intelligence: Harnessing AI Agents to Reinvent Business, Work, and Life*.
- Gohel, Rakesh (@rakeshgohel01) — *AI Agent Progression Framework*.
- Amazon Web Services — arquitectura de Guardrails de Amazon Bedrock; arquitectura de orquestación de agentes basada en grafos.
- Microsoft — Azure AI Content Safety (filtros de entrada/salida); árbol de decisión de Azure AI Foundry para Apps/Agentes.
- McKinsey & Company — estadísticas de adopción de agentes de IA (2025).
- MIT / Fortune — "MIT report: 95% of generative AI pilots at companies are failing" (ago-2025).

Investigación complementaria (añadida en este documento, julio 2026):

- Russell, S. & Norvig, P. — *Artificial Intelligence: A Modern Approach*. Definición formal de agente racional y marco PEAS (Performance, Environment, Actuators, Sensors).
 - Brooks, Rodney — *A Robust Layered Control System for a Mobile Robot* (1986). Origen de la arquitectura reactiva/*subsumption*, contraste histórico del ciclo Sense-Plan-Act deliberativo.
 - Yao, S. et al. — *ReAct: Synergizing Reasoning and Acting in Language Models* (2022). Patrón de razonamiento+acción que formaliza el ciclo Sense-Plan-Act para agentes LLM.
 - Shannon, C. & Weaver, W. — *A Mathematical Theory of Communication* (1948); Jakobson, R. — modelo de funciones del lenguaje (1960). Origen del "ciclo de la comunicación" citado en el material.
 - Chapman & CRISP-DM Consortium — *CRISP-DM 1.0: Step-by-step data mining guide* (2000). Proceso estándar de seis fases, aquí adaptado al ciclo de vida de un agente.
 - Documentación pública de LangGraph, CrewAI, Microsoft AutoGen, Microsoft Semantic Kernel, Auto-GPT, Hugging Face smolagents y AWS Strands Agents (comparativa de paradigmas de orquestación, §13).
 - Society of Automotive Engineers (SAE) — J3016, niveles de automatización de conducción (0-5), usado aquí como analogía pedagógica del framework de 5 niveles de agentes.
-

Documento generado a partir del PDF de la Sesión 8 (Módulo 4, UTEC Posgrado) más investigación propia sobre la definición formal de agente racional, el ciclo de vida CRISP-DM aplicado a agentes, el ecosistema de frameworks de orquestación y el contexto de mercado 2025-2026. Última actualización: 2026-07-14.